Studio Nx

Backend(FastAPI/Python) 60 — update_project()

2026-10-12

Backend(FastAPI/Python) 60 — `update_project()` (제목 · 설명 수정)

항목	내용
파일	`studionx/backend/app/services/project_service.py`
작업	P1-13 프로젝트 API: 프로젝트 정보 수정 (PRD 01 FR-124)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	59 `create_project` · 61 `delete_project`

1. 하는 일

Edit details 모달이 보낸 **제목과 설명 중에서, 실제로 보낸 칸만** 바꿉니다. OWNER와 EDITOR만 수정할 수 있습니다.

보낸 값 (`PATCH` 본문)	결과
`{"title": " New Title "}`	제목은 `New Title`이 되고, 설명은 그대로이며, `updated_at`이 갱신됩니다
`{"description": "new desc"}`	설명만 바뀌고 제목은 그대로입니다
`{"description": null}`, `""`, `" "`	설명을 지웁니다 (None)
`{}`	아무것도 바뀌지 않고, `updated_at`도 그대로입니다
`{"title": "T", "bpm": 200}`	제목만 바뀝니다. BPM은 이 API로 바꿀 수 없습니다 (시퀀서에서 바꿉니다, Phase 2)
VIEWER가 보냄	403 `"You don't have permission."`이고, 아무것도 바뀌지 않습니다
`{"title": null}`, `{"title": ""}`	422입니다. 스키마(56)에서 막혀서 이 함수까지 오지 않습니다

2. 언제 쓰나

호출하는 곳	화면
`PATCH /api/v1/projects/{id}` 라우터 (62): `get_membership`(57)으로 `project`와 `member`를 받은 뒤에 부릅니다	myProject.html 카드의 Edit details 모달에서 Save (P1-14)

라우터는 돌려받은 프로젝트를 `to_item`(58)으로 바꿔서 응답합니다. 그래서 화면은 그 카드 한 장만 다시 그리면 됩니다.

3. 코드

import (파일 맨 위: 59번과 같고, 이번에 새로 추가한 것은 없습니다)

```
from sqlalchemy import func, select
from sqlalchemy.orm import Session

from app.core.errors import AppError, ErrorCode
from app.models import Project, ProjectMember, ProjectSession, Track, User
from app.models.project import MemberRole
```

```
from app.models.track import TrackType
from app.schemas.project import (
    ProjectCreateRequest,
    ProjectItem,
    ProjectListResponse,
    ProjectSort,
    ProjectUpdateRequest,
)
```

이 함수는 `Session`, `Project`, `ProjectMember`, `MemberRole`, `ProjectUpdateRequest`를 사용합니다 (역할 검사는 57번 `require_role`).

함수

```
def update_project(
    db: Session,
    project: Project,
    member: ProjectMember,
    data: ProjectUpdateRequest,
) -> Project:
    require_role(member, MemberRole.OWNER, MemberRole.EDITOR)
    changes = data.model_dump(exclude_unset=True)
    for name, value in changes.items():
        setattr(project, name, value)
    db.commit()
    db.refresh(project)  # 새 updated_at 읽기
    return project
```

문법 풀이

import 문법	뜻
(새로 추가한 import 없음)	59번에서 추가한 `MemberRole`을 역할 검사에 사용합니다

식	뜻
`data.model_dump(exclude_unset=True)`	**본문에 실제로 들어 있던 칸만** 사전으로 꺼냅니다. `{"title": "A"}`를 보내면 `{"title": "A"}`만 나오고 description은 없습니다
`for name, value in changes.items():`	사전의 (이름, 값) 쌍을 하나씩 꺼냅니다
`setattr(project, name, value)`	`project.title = value`와 같습니다. 칸 이름이 변수에 들어 있을 때 씁니다
`require_role(member, MemberRole.OWNER, MemberRole.EDITOR)`	역할이 둘 중 하나가 아니면 403으로 응답합니다 (57)

`exclude_unset`이 중요한 이유:

보낸 본문	`model_dump()`	`model_dump(exclude_unset=True)`
`{"title": "A"}`	`{"title": "A", "description": None}` → 설명이 **지워집니다** ✗	`{"title": "A"}` ✓
`{"description": null}`	`{"title": None, "description": None}`	`{"description": None}` → 설명만 지웁니다 ✓

4. 흐름

1. 역할을 검사합니다. VIEWER라면 403으로 응답하고 아무것도 바꾸지 않습니다.
2. 실제로 보낸 칸만 꺼냅니다.
3. 그 칸만 객체에 넣습니다.
4. `commit`으로 바뀐 칸만 `UPDATE`합니다. 값이 바뀌었다면 DB가 `updated_at`을 지금 시각으로 바꿉니다.
5. `refresh`로 새 `updated_at`을 읽어서 돌려줍니다.

5. 왜 이렇게

선택	이유
`exclude_unset=True`	보내지 않은 칸까지 None으로 덮어쓰면, 제목만 고쳐도 설명이 사라집니다 (PATCH는 일부만 수정하는 요청입니다)
역할 검사를 **맨 앞**에 둠	VIEWER의 요청은 객체를 건드리기 전에 끝냅니다
`setattr`로 반복 처리	바꿀 수 있는 칸이 늘어나도(Phase 2의 키, 스케일 등) 스키마에만 추가하면 됩니다. 스키마에 없는 칸(bpm 등)은 pydantic이 버리기 때문에 들어올 수 없습니다
`updated_at`을 직접 넣지 않음	DB의 `ON UPDATE CURRENT_TIMESTAMP(3)`가 처리합니다. 빈 요청이면 UPDATE가 일어나지 않으므로 시각도 그대로이고, 목록의 "최근 수정순"이 정확하게 유지됩니다
`try/rollback`이 없음	한 행을 UPDATE 한 번만 하므로 반쯤 저장될 일이 없습니다. 실패하면 요청이 끝날 때 세션이 닫히면서 취소됩니다 (59번은 세 행을 저장하므로 필요했습니다)
`title.strip()`과 null 검사가 없음	스키마(56)가 이미 처리합니다 (템플릿에 있던 "title null → 422"를 스키마로 옮겼습니다)

6. 시험 (`tests/test_project_service.py`: 9개를 추가해서 모두 38개)

시험	확인한 것
제목만 수정	공백이 정리된 제목이 저장되고, 설명은 그대로이며, `updated_at`이 10월 1일보다 뒤가 됩니다
설명만 수정	제목은 그대로입니다
설명 지우기 3가지	null, `""`, `" "`는 모두 None이 됩니다
빈 본문	제목, 설명, `updated_at`이 모두 그대로입니다
스키마에 없는 칸	`bpm: 200`은 무시되고 120.00이 유지됩니다
EDITOR	수정할 수 있습니다
VIEWER	403이고, DB의 제목은 그대로입니다

전체 시험 476개가 통과했고, `ruff` 검사도 통과했습니다.